



TypeScript dá forma aos dados do nosso app

Componentes, props, estado e coleções • Aula 4

Técnico em Informática Integrado ao Ensino Médio

Instituto Federal do Maranhão — IFMA

Prof. Thales Levi Azevedo Valente

O ambiente funciona; agora o aplicativo precisa compreender seus dados.



Sem tipos, um dado errado pode chegar até a tela

JAVASCRIPT COMUM

```
const memoria = {  
  id: 1,  
  titulo: "Relato",  
  ano: "dois mil"  
};
```



O que pode dar errado?

Imagine tentar calcular a idade deste registro ou formatar a data para exibição. O erro só apareceria quando o usuário abrisse o app.

Preveja: o que a tela fará com este dado?

O DADO CHEGOU ASSIM

```
const memoria = {  
  titulo: "Relato do Ancião",  
  ano: "dois mil"  
};
```

O campo **ano** deveria representar um número. Mas JavaScript ainda aceitou o texto.

FAÇA UMA PREVISÃO

Se a tela tentar calcular `ano + 1`, qual resultado você espera?

A

O app mostra **2001**.

B

O app mistura texto e número.

C

O editor avisa antes de executar.

Ainda não escolhemos a solução. Primeiro, entendemos o risco.

Previsão: qual opção parece mais provável? Explique com uma frase.

Resultados de Aprendizagem

Ao final desta aula, você será capaz de:



MODELAR

Dados tipados e interfaces



TRANSFORMAR

Coleções em componentes



COMPOR

Interfaces reutilizáveis



INTERAGIR

Usando estado e eventos



TESTAR

Mudanças no navegador/celular



EXPLICAR

A lógica do seu próprio código

Agenda de 100 minutos

0 – 10 min



Retrospectiva verificável da Aula 3

10 – 30 min



Explicação dialogada e demonstração incremental

30 – 85 min



Oficina: uma das duas trilhas práticas

85 – 100
min



Teste, commit, resumo, quiz e próxima aula

O editor pode inferir ou receber uma anotação

1. O VALOR JÁ REVELA O TIPO

```
const ano = 2026;
```

Como o valor é um número, o TypeScript infere:
ano é number.

2. O VALOR CHEGARÁ DEPOIS

```
let titulo: string;  
titulo = "Casa de Farinha";
```

Aqui registramos o tipo antes de conhecer o valor que será usado.

valor conhecido



inferência

OU

anotação explícita

Previsão: se escrevermos `ano = "2026"`, o que o editor deve sinalizar?

Unões restringem escolhas válidas

```
type Categoria =  
    "Relato" | "Lugar" | "Celebração";  
  
const categoria: Categoria = "Relato";
```

O QUE É ISSO?

As **Union Types** permitem que uma variável aceite apenas um conjunto específico de valores fixos.

? PERGUNTA DE PREVISÃO

O que acontece se tentarmos atribuir o valor "Festa" à variável categoria?

Vários valores podem descrever um mesmo registro

UM OBJETO CONCRETO

```
const memoria = {  
  titulo: "Relato do Ancião",  
  comunidade: "Itapecuru",  
  ano: 1950  
};
```

Um **objeto** agrupa informações que pertencem ao mesmo registro.

LEIA UMA LINHA POR VEZ

1

Chave

titulo
nomeia a
informação.

2

Valor

"Relato do
Ancião" é o
conteúdo
guardado.

3

Acesso

memoria.titulo
busca esse
valor.

Ainda é apenas um objeto. O próximo passo será registrar a forma que todos os objetos desse tipo devem seguir.

Verificação: qual expressão acessa o ano? `memoria._____`

Uma interface descreve a forma de um objeto

```
interface Memoria {  
    id: number;  
    titulo: string;  
    comunidade: string;  
    categoria: Categoria;  
    ano?: number;  
}
```

O CONTRATO

A **interface** funciona como um contrato: qualquer objeto que se diga uma "Memoria" deve seguir exatamente essa estrutura.

? PROPRIEDADE OPCIONAL

O símbolo **?** indica que o campo **ano** não é obrigatório. O objeto pode existir sem ele.

Microexercício: o editor aceitaria este objeto?

Considere o contrato: `ano: number` e `titulo: string`.

A

```
{ titulo: "Casa",  
  ano: 1950 }
```

Aceita ou rejeita?

B

```
{ titulo: "Casa",  
  ano: "1950" }
```

Aceita ou rejeita?

C

```
{ titulo: 1950, ano:  
  1950 }
```

Aceita ou rejeita?

Em dupla, marque A, B e C. Depois justifique usando a palavra **tipo**.

Resposta esperada nas notas: A aceita; B e C violam o contrato.

Um registro válido segue o contrato

CONTRATO JÁ DEFINIDO

```
interface Memoria {  
  titulo: string;  
  comunidade: string;  
  ano?: number;  
}
```

O ponto de interrogação permite que ano exista ou não exista.

REGISTRO SIMULADO

```
const memoria1: Memoria = {  
  titulo: "Casa de Farinha",  
  comunidade: "Mirim",  
  ano: 1950  
};
```

O editor compara cada campo do objeto com a interface antes de o dado chegar à tela.

Conexão com o projeto: estes registros são simulados, mas o contrato evita que o app trate cada memória de um jeito diferente.

O próximo problema é repetir a mesma transformação

Agora temos registros válidos. Mas a tela precisa montar uma legenda para cada memória.

MEMÓRIA 1

```
"Relato" + " · " + "Itapecuru"
```

Resultado: **Relato · Itapecuru**

MEMÓRIA 2

```
"Lugar" + " · " + "Mirim"
```

Resultado: **Lugar · Mirim**

O que estamos repetindo: os dados ou a regra de montar a legenda?

No próximo slide, mantemos os mesmos dados e concentramos essa regra em um único lugar.

Repetir a legenda para cada dado custa trabalho

ANTES: FAZEMOS A MESMA REGRA DUAS VEZES

```
const legenda1 =  
  memoria1.categoria + " . " + memoria1.comunidade;  
  
const legenda2 =  
  memoria2.categoria + " . " + memoria2.comunidade;
```

Problema

Se a regra mudar, precisamos lembrar de corrigir cada cópia.

Pista

Os dados mudam. A operação de juntar categoria e comunidade permanece igual.

Previsão: se houver 20 memórias, quantas linhas parecidas teremos de escrever?

Uma função recebe, transforma e devolve

A REGRA EM UM SÓ LUGAR

```
function criarLegenda(memoria) {  
  return memoria.categoria +  
    " . " + memoria.comunidade;  
}
```

Por enquanto, observe só a ideia de **função**. A tipagem virá depois.

LEIA COMO UM FLUXO



A função guarda a regra; cada chamada oferece um dado diferente.

Verificação: o que entra na função? E o que ela devolve?

Existem duas formas de juntar texto e variáveis

CONCATENAÇÃO COM +

```
function criarLegenda(memoria) {  
  return memoria.categoria +  
    " . " + memoria.comunidade;  
}
```

O sinal de + junta pedaços de texto soltos com as variáveis.

TEMPLATE LITERALS COM CRASE

```
function criarLegenda(memoria) {  
  return `${memoria.categoria} .  
    ${memoria.comunidade}`;  
}
```

Usamos **crases** (` `) e colocamos as variáveis dentro de `${ }`.

O resultado é exatamente o mesmo. Em React, o Template Literal facilita muito a leitura quando misturamos texto fixo e variáveis.

Verificação: se usarmos aspas simples (' ') em vez de crases, o `${ }` vai funcionar?

Quem chama a função e o que volta?

```
const legenda = criarLegenda(memoria1);
```

1

Chamada

Esta linha pede que `criarLegenda` execute agora.

2

Entrada

O valor de `memoria1` chega ao parâmetro `memoria`.

3

Retorno

A função devolve um texto; esse texto é guardado em `legenda`.

`memoria1`



`criarLegenda(...)`



"Relato ·
Itapecuru"



`legenda`

Previsão: qual valor entra na segunda chamada `criarLegenda(memoria2)`?

TypeScript melhora o contrato da função

ANTES: JAVASCRIPT

```
function criarLegenda(memoria) {  
  return memoria.categoria +  
    " . " + memoria.comunidade;  
}
```

Funciona, mas não registra o que a função espera nem o que ela devolve.

DEPOIS: TYPESCRIPT

```
function criarLegenda(  
  memoria: Memoria  
): string {  
  return memoria.categoria +  
    " . " + memoria.comunidade;  
}
```

A lógica é a mesma. O contrato agora fica explícito.

O que é novo no lado direito: a regra de legenda ou as informações que o editor pode conferir?

Microexercício: complete o contrato

A função recebe um número e devolve outro número.

```
function dobro(numero: _____): number {  
  return numero * 2;  
}
```

A

string

B

number

C

Memoria

Escolha uma opção e diga por que o operador * ajuda a decidir.

Conexão: no projeto, o mesmo raciocínio protege a função que cria uma legenda.

Repetir a mesma interface não escala

ANTES: JSX REPETIDO MANUALMENTE

```
<View>  
  <Text>Relato do Acião</Text>  
</View>  
  
<View>  
  <Text>Casa de Farinha</Text>  
</View>
```

Limitação

Cada novo registro exige copiar estrutura, estilo e texto.

Pergunta

Se tivermos 20 registros, vamos escrever 20 blocos parecidos?

A próxima ideia resolve a repetição visual: uma função que produz uma parte da interface.

Um componente é uma função que produz interface

COMPONENTE MÍNIMO

```
function CartaoMemoria() {  
  return (  
    <View>  
      <Text>Relato do Ancião</Text>  
    </View>  
  );  
}
```

LEIA A IDEIA

1

É uma função

Tem nome e pode ser chamada.

2

Retorna JSX

A saída descreve uma parte da interface.

3

Começa com maiúscula

CartaoMemoria sinaliza um componente.

Por enquanto, este componente ainda está fixo: sempre mostra o mesmo texto.

Verificação: qual trecho é a saída da função? O que o usuário verá na tela?

Mesmo componente, partes visuais repetidas

CHAMAMOS DUAS VEZES

```
<CartaoMemoria />  
<CartaoMemoria />
```

O componente evita copiar toda a estrutura JSX.

MAS A TELA RESULTANTE

Relato do A ancião

Relato do A ancião

As duas instâncias ainda são iguais.

mesmo componente

+

sem entrada diferente

→

mesma tela

Como usar o mesmo componente para mostrar “Casa de Farinha” sem reescrever a função?

Uma função comum já recebe entrada

FUNÇÃO CONHECIDA

```
function cumprimentar(nome) {  
  return "Olá " + nome;  
}  
  
cumprimentar("Ana");
```

O valor "Ana" entra pelo parâmetro nome.

ANALOGIA PARA COMPONENTES

valor



componente



JSX

Uma **prop** é uma entrada que o componente recebe para produzir uma interface diferente.

Função

parâmetro → retorno

Componente

prop → JSX

Importante

É uma analogia para pensar, não uma identidade perfeita.

Se nome muda na função, o cumprimento muda. O que pode mudar quando uma prop muda?

O cartão fixo não mostra outro título

A FUNÇÃO ESTÁ ENGESSADA

```
function CartaoMemoria() {  
  return (  
    <Text>Relato do Ancião</Text>  
  );  
}
```

O texto está escrito dentro do componente.

QUEREMOS DUAS TELAS

Relato do Ancião

Casa de Farinha

Mas duas chamadas de
<CartaoMemoria /> continuam
mostrando o primeiro título.

Qual valor o componente precisa receber para deixar de ser fixo?

A prop é a entrada de um componente

O COMPONENTE RECEBE

```
function CartaoMemoria(props) {  
  return (  
    <Text>{props.titulo}</Text>  
  );  
}
```

props.titulo busca o valor que chegou ao componente.

QUEM CHAMA ENTREGA O VALOR

```
<CartaoMemoria  
  titulo="Relato do Ancião"  
>
```

"Relato..."



props.titulo



<Text>

Previsão: qual texto aparece na tela depois dessa chamada?

Uma prop é de leitura: o cartão usa o valor recebido; ele não deve reescrever esse valor.

Props diferentes produzem telas diferentes

INSTÂNCIA 1

```
<CartaoMemoria  
  titulo="Relato do A ancião"  
>
```

Relato do A ancião

INSTÂNCIA 2

```
<CartaoMemoria  
  titulo="Casa de Farinha"  
>
```

Casa de Farinha

mesmo componente

+

props diferentes

→

interfaces diferentes

O que permanece igual nas duas chamadas? O que muda?

Também podemos registrar o tipo da prop

PROP SEM CONTRATO

```
function CartaoMemoria(props) {  
  return <Text>{props.titulo}  
</Text>;  
}
```

O componente recebe uma prop, mas o editor não sabe qual formato esperamos.

PROP COM CONTRATO

```
type CartaoProps = {  
  titulo: string;  
};  
  
function CartaoMemoria(props:  
  CartaoProps) {  
  return <Text>{props.titulo}  
</Text>;  
}
```

Agora o editor exige um título de texto.

O comportamento visual do cartão mudou? Ou o editor ganhou uma regra para conferir?

Um objeto inteiro evita listas longas de props

SEPARAMOS CADA CAMPO

```
<CartaoMemoria  
  titulo={memoria.titulo}  
  comunidade={memoria.comunidade}  
  ano={memoria.ano}  
>
```

Funciona, mas a lista cresce quando a memória ganha novos campos.

ENVIAMOS O REGISTRO

```
type CartaoProps = {  
  memoria: Memoria;  
};  
  
<CartaoMemoria memoria={memoria} >
```

A prop agora carrega todo o objeto que já segue o contrato `Memoria`.

Qual opção reduz a repetição quando o objeto recebe uma nova propriedade?

No próximo slide, veremos como acessar um campo dentro dessa prop-objeto.

Desestruturar simplifica a leitura, não muda o comportamento

ANTES

```
function Cartao(  
  props: CartaoProps  
) {  
  return (  
    <Text>  
      {props.memoria.titulo}  
    </Text>  
  );  
}
```

Acessamos o objeto inteiro por props.memoria.

DEPOIS

```
function Cartao(  
  { memoria }: CartaoProps  
) {  
  return (  
    <Text>  
      {memoria.titulo}  
    </Text>  
  );  
}
```

Retiramos memoria de props logo na entrada.

Previsão: o texto que aparece na tela muda? Por quê?

Resposta esperada: não. Apenas encurtamos o caminho para ler a mesma prop.

Microexercício: identifique componente, prop e valor

```
<CartaoMemoria titulo="Relato" />
```

1

Componente

Qual parte nomeia a função que produz interface?

2

Prop

Qual parte nomeia a entrada entregue ao componente?

3

Valor

Qual parte é o dado concreto usado nesta chamada?

Em dupla, apontem no código: componente = ____; prop = ____; valor = ____.

Resposta esperada: CartaoMemoria; titulo; "Relato".

Um array organiza vários valores

UMA LISTA EM UMA VARIÁVEL

```
const nomes = [  
  "Ana",  
  "Bruno",  
  "Carla"  
];
```

Um **array** é uma coleção ordenada de valores.

PODEMOS ACESSAR UMA POSIÇÃO

0

nomes[0] →
"Ana"

1

nomes[1] →
"Bruno"

2

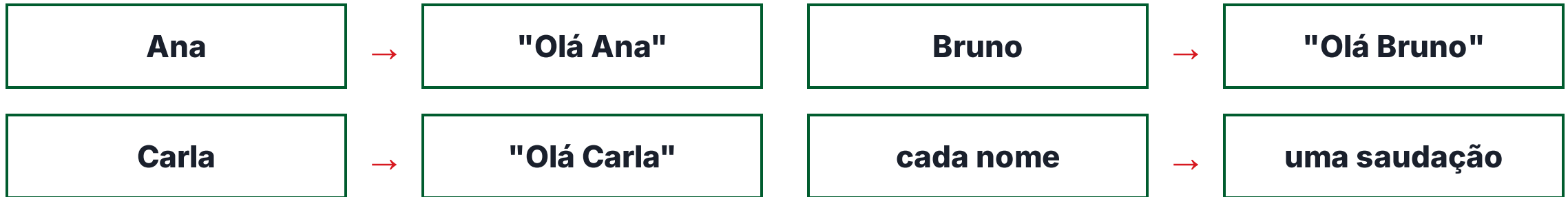
nomes[2] →
"Carla"

A contagem começa em 0. O array guarda os itens; ainda não fizemos transformação alguma.

Previsão: qual valor aparece em nomes[2]?

Cada item pode gerar outro valor

TRANSFORMAR NÃO É APENAS COPIAR



Entrada

Um nome vindo do array.

Regra

Montar o texto "Olá " + nome.

Saída

Uma saudação para aquele nome.

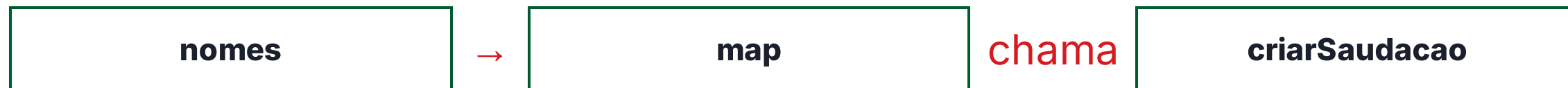
Se o item for "Carla", qual deve ser a saída da transformação?

Uma função também pode ser entregue para outra função

UMA FUNÇÃO CONHECIDA

```
function criarSaudacao(nome) {  
  return "Olá " + nome;  
}
```

Ela recebe um nome e devolve uma saudação.



AGORA UMA IDEIA NOVA

```
nomes.map(criarSaudacao);
```

Aqui não executamos `criarSaudacao()` agora. Entregamos a função para `map` usar depois.

Qual é a função entregue para `map`? O que mudaria se escrevêssemos parênteses?

Arrow function é outra escrita para uma função

FORMA TRADICIONAL

```
function dobro(numero) {  
  return numero * 2;  
}
```

Nome, parâmetro, regra e retorno aparecem em blocos separados.

FORMA COM ARROW

```
const dobro = (numero) => {  
  return numero * 2;  
};
```

A mesma regra pode ser guardada em uma constante.

FORMA CURTA, QUANDO HÁ UM ÚNICO RETORNO

```
const dobro = (numero) => numero * 2;
```

O que permanece igual nas três versões: a função ou apenas a forma de escrevê-la?

.map() aplica uma transformação a todos os itens

```
const saudacoes = nomes.map(criarSaudacao);
```

Para cada nome, map chama a função que recebeu.

Ana → criarSaudacao → "Olá Ana"

Bruno → criarSaudacao → "Olá Bruno"

Carla → criarSaudacao → "Olá Carla"



Quantas vezes `criarSaudacao` será chamada para este array?

O callback pode ser escrito com arrow function

CALLBACK NOMEADO

```
nomes.map(  
  criarSaudacao  
);
```

A função foi criada em outro lugar e entregue ao map.

CALLBACK ESCRITO AQUI

```
nomes.map(  
  (nome) => "Olá " + nome  
);
```

A arrow recebe cada nome e devolve a saudação correspondente.

nome

é o item atual do array.

=>

separa entrada e regra.

"Olá " + nome

é o valor devolvido.

Na segunda execução, qual valor chega no parâmetro nome?

Agora cada item é um objeto Memoria

ARRAY DE OBJETOS

```
const memorias = [  
  { titulo: "Relato" },  
  { titulo: "Casa" }  
];
```

Cada posição guarda um objeto, não apenas um texto.

TRANSFORMAÇÃO DE CADA OBJETO

```
memorias.map(  
  (memoria) =>  
    memoria.titulo  
);
```

Em cada execução, `memoria` representa um objeto do array.

`{ titulo: "Relato" }`



`memoria.titulo`



`"Relato"`

Previsão: na segunda execução, o que `memoria.titulo` devolve?

O resultado da transformação pode ser JSX

NO SLIDE ANTERIOR

```
memorias.map(  
  (memoria) =>  
    memoria.titulo  
);
```

Cada item gerava um texto.

AGORA

```
memorias.map(  
  (memoria) => (  
    <Text>{memoria.titulo}</Text>  
  )  
);
```

Cada item gera uma parte da interface.

objeto Memoria



JSX



texto na tela

Esse map devolve dados ou componentes visuais?

Dados viram instâncias do cartão

A TRANSFORMAÇÃO COMPLETA

```
memorias.map(  
  (memoria) => (  
    <CartaoMemoria  
    memoria={memoria}  
  />  
)  
);
```

LEITURA PASSO A PASSO

1

map percorre
as memórias.

2

O item atual
entra em
memoria.

3

O objeto vira
uma prop do
cartão.

A tela recebe um cartão por memória; a estrutura do cartão continua reutilizável.

O que chega na prop `memoria` quando o loop está no segundo registro?

A lista precisa identificar cada item

O QUE REACT ENXERGA

Cartão: Relato

Cartão: Casa

Cartão: Lugar

São três cartões parecidos. Se um item mudar de posição, como saber qual é qual?

O AVISO DO EDITOR

"Each child in a list should have a unique key prop."

Não é aparência

Não muda a cor nem o texto do cartão.

É identidade

Ajuda o React a acompanhar cada item ao atualizar a lista.

Qual campo de uma memória poderia funcionar como identidade única: título ou id?

id é a identidade estável do item

APENAS UMA NOVIDADE NO CARTÃO

```
<CartaoMemoria  
  key={memoria.id}  
  memoria={memoria}  
>
```

POR QUE ID?

1

É um identificador do registro.

2

Não depende da posição no array.

3

Permanece igual se a lista for reordenada.

EVITE ESTA ALTERNATIVA

```
key={indice}
```

O índice pode apontar para outro item quando a lista muda.

A key não é uma prop comum do cartão; ela ajuda o React a gerenciar a lista.

Se “Casa” passar do índice 1 para o índice 0, qual informação continua estável: índice ou id?

Uma variável comum muda; a tela pode não mudar

UMA VARIÁVEL JAVASCRIPT

```
let selecionado = false;  
selecionado = true;
```

O valor na memória mudou de false para true.

O PROBLEMA EM REACT



Mudar uma variável não é o mesmo que solicitar uma nova renderização.

Se o usuário toca no cartão, como React saberia que deve redesenhar a tela?

Estado é o dado que a interface precisa lembrar

Estado é um dado que o componente precisa lembrar e cuja mudança pode alterar o que aparece na tela.

usuário escolhe
um cartão



o app lembra
a escolha



a tela destaca
o cartão

NÃO É ESTADO

Um texto fixo que nunca muda, como o título da aula.

PODE SER ESTADO

Qual cartão está selecionado agora.

Qual informação o nosso cartão precisa lembrar depois do toque?

useState(false) cria valor, setter e inicial

```
const [selecionado, setSelecionado] =  
  useState(false);
```

1

Valor atual

selecionado guarda o estado agora.

2

Setter

setSelecionado solicita a mudança.

3

Inicial

false informa como a tela começa.

A diferença essencial: o setter avisa React que uma mudança relevante aconteceu.

Antes de qualquer toque, qual é o valor de selecionado?

Leia cada parte da linha de estado

```
const [selecionado, setSelecionado] = useState(false);
```

SELECIONADO

valor atual

Informa o que o componente lembra agora.

SETSELECIONADO

função de mudança

Pede uma atualização do estado.

FALSE

valor inicial

Define como o estado começa antes da interação.

valor atual

+

setter

+

valor inicial

Se chamarmos o setter com true, qual valor o componente lembrará depois?

Setter pede uma nova renderização

```
setSelecionado(true);
```

evento



setter



estado muda



React
renderiza



tela muda

Variável comum

O valor muda sem notificar a interface.

Setter

A mudança é comunicada e o componente é atualizado.

Qual parte do fluxo faz a tela ser redesenhada: o valor antigo ou a chamada do setter?

Entregamos uma função para o toque executar depois

PRIMEIRO, A AÇÃO NOMEADA

```
function selecionar() {  
  setSelected(true);  
}
```

A função já sabe qual mudança de estado solicitar.

DEPOIS, ENTREGAMOS A FUNÇÃO

```
<Pressable  
  onPress={selecionar}  
>
```

React guarda essa função e a executa quando o toque acontecer.

ERRADO NESTE CASO

```
onPress={selecionar()}
```

Executa agora, durante a renderização.

CERTO

```
onPress={selecionar}
```

Entrega a função para executar depois.

Quando a função correta deve executar: agora ou quando o usuário tocar?

A arrow cria a função que usará aquele id depois

A AÇÃO AGORA PRECISA DE UM DADO

```
function selecionar(id) {  
  setIdSelecionada(id);  
}
```

Cada cartão tem seu próprio id.

ENTREGAMOS UMA FUNÇÃO NOVA

```
<Pressable  
  onPress={() =>  
    selecionar(memoria.id)  
  }  
>
```

A arrow não chama selecionar agora. Ela cria uma função para o toque executar depois.

toque no cartão



arrow executa



selecionar(id)



setter

Qual id chegará a selecionar se o usuário tocar no cartão "Casa"?

Dados, props, eventos e estado trabalham juntos

OS DADOS DESCEM



Cada cartão recebe o objeto que deve mostrar.

A INTERAÇÃO ALTERA A TELA



O estado guarda qual escolha deve aparecer agora.

Dados descrevem o conteúdo; props transportam o conteúdo; eventos e estado controlam a reação da interface.

Em qual caminho aparece `memoria.id`: no dado que desce ou no toque que pede seleção?

Filtrar é decidir quem fica na lista

```
const numeros = [2, 7, 10, 3];  
const maiores = numeros.filter(  
  (numero) => numero ≥ 5  
);
```

2 → false → sai

7 → true → fica

10 → true → fica

3 → false → sai

[2, 7, 10, 3]

→ filter

[7, 10]

Previsão: o número 5 ficaria ou sairia com esta regra?

Filter reduz a lista; map cria a interface

INDICADORES SIMULADOS

```
[{ nome: "Frequência", situacao:
  "critica" },
 { nome: "AVA", situacao: "estavel" }]
```

Sem dados pessoais reais: usamos apenas indicadores didáticos agregados.

CADEIA DE TRABALHO

```
indicadores
  .filter(i => i.situacao ==
    "critica")
  .map(i => <Cartao indicador={i} />)
```

filter decide quem fica; **map** transforma quem ficou em componente.

todos os indicadores

→ filter

só críticos

→ map

cartões na tela

Se um indicador está "estável", em qual etapa ele deixa de seguir para a tela crítica?

Cada checkpoint precisa funcionar antes do próximo

Em dupla, escolham Memórias ou EDA. Não avancem por cópia: testem cada etapa antes de continuar.

1

Projeto abre

Executem a atividade e confirmem a tela inicial.

2

Tipagem

Resolvam interface e tipos; validem o editor.

3

Cartão

Façam o componente mostrar a prop recebida.

4

Coleção

Useem map e verifiquem os cartões.

5

Interação

Implementem estado e evento; testem o toque.

6

Evidência

Capturem screenshot e registrem o commit.

Regra da oficina: se um checkpoint falhar, a dupla pede ajuda com o erro e o checkpoint atual.

Escolha a trilha e siga o fluxo do repositório

1. ENTRE NO PROJETO

```
git clone <repositorio>
cd lessons/2/memory-project
# ou lessons/2/eda-project
npm ci
npx expo-doctor
npx expo start
```

w abre no navegador; o QR Code abre no celular.
Arquivo principal: app/index.tsx.

2. ESCOLHA SUA ENTREGA

Memórias

Cartões tipados,
coleção, seleção e
detalhes simulados.

EDA

Indicadores simulados,
filter, map e estado de
filtro.

**Saída: screenshot do app funcionando +
commit descritivo.**

```
git commit -m "feat: implement typed  
memory cards"
```

Referências oficiais: Microsoft TypeScript Handbook; React e React Native Documentation (Meta); Expo Documentation. Acesso em 18 ago. 2026. Referências ABNT completas: notas do apresentador.